

Elegimos Python 3.11 como lenguaje principal y Guppy como lenguaje cuántico para implementar QAOA. Python permitió integrar el procesamiento geoespacial, la construcción de grafos, la optimización clásica, el análisis estadístico y la generación reproducible de resultados. Guppy permitió definir las capas de costo y mezcla, las mediciones y la compilación de los circuitos a HUGR.

Utilizamos Quantinuum Nexus para gestionar la ejecución remota y SelenePlus como backend de emulación con un modelo de ruido. La integración permitió compilar, enviar y recuperar los resultados de las cuatro instancias. Sin embargo, SelenePlus no es hardware cuántico físico y sus tiempos no pueden utilizarse como evidencia de aceleración.

La separación entre el circuito cuántico, la optimización clásica y el análisis de resultados funcionó adecuadamente. También fue posible automatizar el proceso mediante un único punto de entrada reproducible.

Las principales dificultades fueron la sensibilidad de QAOA a la inicialización, el aumento del costo lógico con la profundidad y la degradación observada bajo ruido. Una sola ejecución remota por instancia no permite caracterizar estadísticamente el backend. Consideramos que faltan herramientas más directas para repetir experimentos remotos, analizar detalladamente el ruido e incorporar técnicas de mitigación. El SDK fue funcional, pero requirió una orquestación clásica considerable.